

外部任务行为与姿态数据接口说明

适用于 PLATFORM_CONTROL 与 WEAPON_FIRING 外部任务

协议版本：1.0
传输方式：WebSocket
任务关联：服务端解析 task_runs.task_id
时间基准：Unix 毫秒时间戳

1. 文档目的

本文档定义“平台控制”和“武器发射”两类外部任务的行为数据、姿态数据及其任务关联规则，用于外部仿真平台与 F18-Radar 服务端联调、数据落库和后续实验分析。

适用任务类型：

任务	task_type
平台控制	PLATFORM_CONTROL
武器发射	WEAPON_FIRING

本文档约定：

- 一次按钮按下或控制操作生成一条行为事件。
- 行为事件记录操作者、任务类型、按键、行为含义和发生时间。
- 行为事件同时携带操作发生时的当前姿态快照。
- 外部客户端可以不提供 task_id，由服务端根据当前活动任务解析。
- 所有客户端业务时间戳统一使用 Unix 毫秒时间戳。

2. 通信方式

行为和姿态数据通过现有 WebSocket 连接发送。

为避免行为字段 `type` 与 WebSocket 消息类型冲突，消息采用“外层消息类型 + 内层业务数据”的结构：

```
JSON
{
  "type": "external_behavior_record",
  "data": {
    "type": "Fire"
  }
}
```

其中：

- 外层 `type` 表示 WebSocket 消息类型。
- `data.type` 表示本次行为的业务类型。

3. 通用约定

3.1 字段命名

外部协议已有的 `PlayerID` 字段继续保留，服务端接收后统一映射为内部 `user_id`。

原始草案中的 `"pose data"` 统一规范为合法、稳定的 JSON 字段名：

```
TEXT
pose_data
```

3.2 时间单位

客户端发送的以下字段均使用 Unix 毫秒时间戳：

- 行为事件 `timestamp`
- 姿态记录 `timestamp`
- 姿态快照 `captured_at`

示例：

```
JSON
{
  "timestamp": 1720000000000
}
```

服务端可以另外记录微秒级接收时间，但不得覆盖客户端原始时间。

3.3 任务类型

接口仅接受以下大写枚举值：

```
TEXT
PLATFORM_CONTROL
WEAPON_FIRING
```

现有代码中的 `platform_control`、`weapon_launch` 仅作为内部路由分类，不作为本接口对外值。

3.4 task_id 规则

`task_id` 为可选字段。

外部客户端可以发送：

```
JSON
{
  "task_id": null
}
```

当 `task_id` 缺失或为 `null` 时，服务端必须使用以下条件解析当前具体子任务：

```
TEXT
user_id = PlayerID
task_type = PLATFORM_CONTROL 或 WEAPON_FIRING
status = active
```

解析顺序：

1. 优先使用外部任务内存上下文中的 `gaze_task_id` 或 `current_subtask_task_id`。
2. 内存上下文不存在时，查询 `task_runs` 中相同用户、相同任务类型的活动记录。
3. 只找到一条记录时，使用该记录的 `task_id`。
4. 找到多条且无法通过当前上下文消歧时，拒绝写入。
5. 未找到活动任务时，拒绝写入。

数据库最终保存的必须是具体子任务 `task_runs.task_id`，不得使用 `task_groups.group_id` 代替。

4. 行为数据记录接口

4.1 接口语义

行为数据记录表示：

某个用户或 AI 在某个具体任务下，于某一时刻执行了一次按键或控制操作，并同时保存该时刻的当前姿态数据。

按钮持续按住时只记录一次“未按下到按下”的状态变化，不得在轮询期间重复生成相同行为记录。

4.2 完整请求示例

```
JSON
{
  "type": "external_behavior_record",
  "schema_version": "1.0",
  "client_event_id": "7d72c9dc-2674-483e-bf10-c77035379a56",
  "data": {
    "PlayerID": "P001",
    "task_type": "PLATFORM_CONTROL",
    "task_id": null,
    "button": "button1",
    "type": "Fire",
    "timestamp": 1720000000000,
    "owner": "User",
    "pose_data": [
      {
        "Side": "Our",
        "ID": "101",
        "timestamp": 1720000000000,
        "posture": {
          "X": 1200.25,
          "Y": 530.50,
          "Z": 820.00
        }
      },
      {
        "Side": "Enemy",
        "ID": "201",
        "timestamp": 1720000000000,
        "posture": {
          "X": 4280.10,
          "Y": 920.30,
          "Z": 760.40
        }
      }
    ]
  }
}
```

4.3 顶层字段

字段	类型	必填	说明
type	string	是	固定为 external_behavior_record
schema_version	string	是	当前固定为 1.0
client_event_id	string/UUID	是	客户端事件唯一 ID，用于幂等写入
data	object	是	行为事件业务数据

4.4 data 字段

字段	类型	必填	说明
PlayerID	string	是	当前用户/被试 ID，服务端映射为 user_id
task_type	string	是	PLATFORM_CONTROL 或 WEAPON_FIRING
task_id	string/integer/ null	否	具体子任务 ID；为空时由服务端解析
button	string/null	条件必填	按钮来源事件填写 button1 ~ button6；非按钮轴事件可为 null
type	string	是	行为类型
timestamp	integer	是	行为发生时间，Unix 毫秒
owner	string	是	本次行为的执行方：User 或 AI
pose_data	array	是	行为发生时的当前姿态快照；无实体时发送空数组

4.5 button 枚举

TEXT
button1
button2
button3
button4
button5
button6

按钮编号表示物理按钮编号，具体按钮与业务行为之间的映射由外部任务配置决定。

4.6 行为类型枚举

枚举值	中文含义
Fire	开火
LeftPedal	左踏板
RightPedal	右踏板
Accelerator	油门
SwitchMode	切换模式
Joystick	操纵杆
OKbutton	确认按钮

当前版本只记录控制行为的发生。若后续需要记录油门比例、踏板行程或操纵杆二维轴值，应新增独立的 `value` 字段，不得把数值拼接进 `button` 或 `type`。

4.7 执行方枚举

枚举值	中文含义	内部兼容值
User	用户执行	manual
AI	AI 执行	AI

对外协议必须保留 `User/AI`；如复用现有 `user_operations.event_owner`，服务端负责将 `User` 转换为 `manual`。

4.8 AI 行为示例

```
JSON
{
  "type": "external_behavior_record",
  "schema_version": "1.0",
  "client_event_id": "13907979-eed5-413c-8910-d2db4248c18a",
  "data": {
    "PlayerID": "P001",
    "task_type": "WEAPON_FIRING",
    "button": "button1",
    "type": "Fire",
    "timestamp": 1720000003000,
    "owner": "AI",
    "pose_data": [
      {
        "Side": "Our",
        "ID": "101",
        "timestamp": 1720000003000,
        "posture": {
          "X": 1320.25,
          "Y": 580.50,
          "Z": 810.00
        }
      },
      {
        "Side": "Enemy",
        "ID": "201",
        "timestamp": 1720000003000,
        "posture": {
          "X": 3210.40,
          "Y": 810.20,
          "Z": 790.00
        }
      }
    ]
  }
}
```

5. 姿态数据结构

5.1 接口语义

姿态数据表示某个实体在某一时刻的当前位置。

行为事件中的 `pose_data` 是操作发生时的快照，不是指向后续可变状态的引用。行为记录写入后，对应姿态快照不得被后续位置更新覆盖。

5.2 姿态对象

```
JSON
{
  "Side": "Our",
  "ID": "101",
  "timestamp": 1720000000000,
  "posture": {
    "X": 1200.25,
    "Y": 530.50,
    "Z": 820.00
  }
}
```

5.3 姿态字段

字段	类型	必填	说明
Side	string	是	实体所属阵营：Our 或 Enemy
ID	string	是	实体唯一 ID
timestamp	integer	是	本条姿态的采集时间，Unix 毫秒
posture	object	是	当前三维坐标
posture.X	number	是	X 轴坐标
posture.Y	number	是	Y 轴坐标
posture.Z	number	是	Z 轴坐标

posture 必须使用结构化数值对象，不得使用 "x,y,z" 拼接字符串。

5.4 阵营枚举

枚举值	中文含义
Our	我方
Enemy	敌方

同一姿态快照中允许存在多个我方或敌方实体，每个实体使用独立 ID。

5.5 坐标系与单位

外部仿真平台必须保证同一任务中的 X、Y、Z 使用相同坐标系和单位。

如果后续需要支持不同仿真环境，建议在姿态快照中增加：

```
JSON
{
  "coordinate_system": "由外部平台定义",
  "position_unit": "由外部平台定义"
}
```

在坐标系和单位尚未正式约定前，服务端只负责原值保存，不对坐标进行单位换算。

6. 独立姿态快照接口

行为记录已经携带当前姿态数据。若外部平台还需要在没有行为事件时独立记录姿态变化，可以使用本接口。

6.1 请求示例

```
JSON
{
  "type": "external_pose_record",
  "schema_version": "1.0",
  "client_snapshot_id": "366dce79-7610-4d3d-b335-e6d126fe27a4",
  "data": {
    "PlayerID": "P001",
    "task_type": "PLATFORM_CONTROL",
    "task_id": null,
    "captured_at": 1720000000000,
    "pose_data": [
      {
        "Side": "Our",
        "ID": "101",
        "timestamp": 1720000000000,
        "posture": {
          "X": 1200.25,
          "Y": 530.50,
          "Z": 820.00
        }
      },
      {
        "Side": "Enemy",
        "ID": "201",
        "timestamp": 1720000000000,
        "posture": {
          "X": 4280.10,
          "Y": 920.30,
          "Z": 760.40
        }
      }
    ]
  }
}
```

6.2 字段

字段	类型	必填	说明
type	string	是	固定为 external_pose_record
schema_version	string	是	当前固定为 1.0
client_snapshot_id	string/UUID	是	客户端姿态快照唯一 ID
data.PlayerID	string	是	当前用户/被试 ID
data.task_type	string	是	PLATFORM_CONTROL 或 WEAPON_FIRING
data.task_id	string/integer/ null	否	具体子任务 ID；为空时由服务端解析
data.captured_at	integer	是	整组姿态快照采集时间，Unix 毫秒
data.pose_data	array	是	当前姿态对象数组

独立姿态接口用于低频状态快照。若需要持续高频记录原始仿真姿态流，应另行定义采样频率、批量格式、压缩和文件存储方案，避免高频逐条写入主 SQLite 数据库。

7. 行为与姿态关联规则

7.1 原子记录

推荐流程：

TEXT

行为发生

→ 生成 client_event_id

→ 读取当前姿态

→ 形成 pose_data 快照

→ 一次发送 external_behavior_record

→ 服务端在同一事务中写入行为和姿态

行为数据和姿态数据必须关联到同一个解析后的 task_id。

7.2 时间关系

正常情况下：

TEXT

行为 timestamp ≈ 姿态 timestamp

如果外部平台不能同时采集，应选择行为发生时刻之前最近的一份有效姿态，不得使用行为发生后的未来姿态冒充当前姿态。

7.3 幂等规则

- `client_event_id` 在行为事件中必须唯一。
- `client_snapshot_id` 在独立姿态快照中必须唯一。
- 服务端收到重复 ID 时返回原写入结果，不得重复插入。

8. 成功响应

8.1 行为记录成功

JSON

```
{
  "type": "external_behavior_record_result",
  "ok": true,
  "client_event_id": "7d72c9dc-2674-483e-bf10-c77035379a56",
  "behavior_record_id": 1001,
  "resolved_task_id": 42,
  "task_type": "PLATFORM_CONTROL",
  "pose_record_count": 2,
  "duplicate": false,
  "server_time_us": 1720000000123456
}
```

8.2 姿态记录成功

JSON

```
{
  "type": "external_pose_record_result",
  "ok": true,
  "client_snapshot_id": "366dce79-7610-4d3d-b335-e6d126fe27a4",
  "pose_snapshot_id": 2001,
  "resolved_task_id": 42,
  "task_type": "PLATFORM_CONTROL",
  "pose_record_count": 2,
  "duplicate": false,
  "server_time_us": 1720000000123456
}
```

9. 错误响应

```
JSON
{
  "type": "external_behavior_record_result",
  "ok": false,
  "client_event_id": "7d72c9dc-2674-483e-bf10-c77035379a56",
  "error": {
    "code": "ACTIVE_TASK_NOT_FOUND",
    "message": "未找到当前用户和任务类型对应的活动子任务"
  }
}
```

建议错误码：

错误码	含义
INVALID_PAYLOAD	JSON 结构或字段类型错误
UNSUPPORTED_TASK_TYPE	不支持的任务类型
UNSUPPORTED_BEHAVIOR_TYPE	不支持的行为类型
INVALID_BUTTON	按钮编号不合法
INVALID_OWNER	owner 不是 User 或 AI
INVALID_TIMESTAMP	时间戳缺失或不是毫秒整数
INVALID_POSE_DATA	姿态结构或坐标值不合法
TASK_ID_MISMATCH	客户端 task_id 与当前活动任务不一致
ACTIVE_TASK_NOT_FOUND	未找到匹配的活动子任务
ACTIVE_TASK_AMBIGUOUS	找到多条活动子任务且无法消歧
INTERNAL_ERROR	服务端写入失败

10. 数据库存储建议

10.1 行为数据

行为事件可以复用现有 user_operations:

接口字段	user_operations
服务端解析出的 task_id	task_id
data.type	operation_type
data.timestamp	timestamp
服务端接收时间	receive_timestamp
PlayerID	user_id
owner=User	event_owner>manual
owner=AI	event_owner=AI
button、原始 owner、client_event_id 等	parameters JSON

如果需要对外部行为做独立统计，也可以新增 external_behavior_records，但不得同时把同一事件重复计入两套业务统计。

10.2 姿态数据

建议新增独立姿态表 `external_pose_records`，每个实体一行：

字段	说明
<code>id</code>	自增主键
<code>pose_snapshot_id</code>	姿态快照 ID
<code>behavior_operation_id</code>	关联 <code>user_operations.id</code> ，独立快照时可为空
<code>client_snapshot_id</code>	独立姿态快照客户端 ID
<code>task_id</code>	关联 <code>task_runs.task_id</code>
<code>user_id</code>	用户 ID
<code>task_type</code>	PLATFORM_CONTROL 或 WEAPON_FIRING
<code>side</code>	Our 或 Enemy
<code>entity_id</code>	实体 ID
<code>pose_time_ms</code>	姿态采集时间
<code>x、y、z</code>	三维坐标
<code>coordinate_system</code>	坐标系，可为空
<code>position_unit</code>	位置单位，可为空
<code>raw_payload_json</code>	原始姿态对象
<code>created_at</code>	数据库写入时间

建议索引：

```
TEXT
(task_id, pose_time_ms)
(task_id, side, entity_id, pose_time_ms)
(behavior_operation_id)
```

10.3 事务要求

处理 `external_behavior_record` 时，行为事件和其 `pose_data` 必须在同一数据库事务中写入：

TEXT

行为写入成功 + 所有姿态写入成功 → 提交

任一写入失败 → 全部回滚

不得出现行为记录已保存、对应姿态只保存一部分的状态。

11. 联调检查清单

- ☐ `task_type` 使用 `PLATFORM_CONTROL` 或 `WEAPON_FIRING`
- ☐ `PlayerID` 与外部任务启动消息中的 `ID` 一致
- ☐ `task_id` 缺失时能从当前活动 `task_runs` 解析
- ☐ 行为只在按下边沿记录一次
- ☐ `timestamp` 为 Unix 毫秒
- ☐ `owner` 只使用 `User` 或 `AI`
- ☐ `button` 使用 `button1 ~ button6` 或 `null`
- ☐ `pose_data` 是数组
- ☐ `posture` 使用数值对象，不使用字符串
- ☐ 行为与姿态写入同一个具体子任务
- ☐ 重复 `client_event_id` 不产生重复数据
- ☐ 成功响应返回 `resolved_task_id`